Chronos (Android) — All Bugs + Fixes

Source reviewed: `Chronos-Native-Android-Project.zip` (16 files).
Status of every item: *Confirmed by reading the code.* Nothing was built or run, because the project is missing files (see C-01). The website (chronos-task-planne.ai.studio) could not be tested, so it is not covered here.
Priority key: P1 = broken core / won't build, P2 = important defect, P3 = polish. No P0 found.

Summary table

ID	Problem	Priority	Effort
C-01	Project missing resource / build files	P1	20 min
C-02	`Theme.kt` missing `Color` import	P1	1 min
C-03	Invalid time text crashes alarm scheduling	P1	20 min
C-04	Reminder silently skipped for near/past times	P2	15 min
C-05	Snooze can crash and loses task info	P2	15 min
C-06	Tasks tab toggle ignores alarms / `completedAt`	P2	10 min
C-07	Notification "Complete" may not save	P2	10 min
C-08	Priority cannot be chosen	P2	10 min
C-09	Home list shows completed tasks; 100% when empty	P3	5 min
C-10	Task ID collision can overwrite a task	P2	5 min
C-11	`fallbackToDestructiveMigration` wipes data	P2 (before release)	15 min
C-12	No way to grant exact-alarm permission; risky permission	P2	15 min
C-13	Alarms lost after app update / timezone change	P2	15 min
C-14	"Native Compose" dev badge in UI	P3	1 min
C-15	README overclaims; unused WorkManager	P3	5 min
C-16	No delete / edit / empty state	P2	1–2 hrs
C-17	Accessibility gaps	P3	20 min

C-01 — Missing resource and build files (P1)

Problem: The manifest references `@style/Theme.Chronos`, `@mipmap/ic_launcher`, `@mipmap/ic_launcher_round`, `@xml/backup_rules` and `@xml/data_extraction_rules`. `app/build.gradle.kts` references `proguard-rules.pro`. None of these are in the zip. There's also no `gradlew`, and the root `build.gradle.kts` / `settings.gradle.kts` were not reviewed.

Fix:

`app/src/main/res/values/themes.xml`

```
<resources>
    <style name="Theme.Chronos" parent="android:Theme.Material.NoActionBar" />
</resources>
```

`app/src/main/res/xml/backup_rules.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<full-backup-content />
```

`app/src/main/res/xml/data_extraction_rules.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<data-extraction-rules>
    <cloud-backup />
    <device-transfer />
</data-extraction-rules>
```

`app/proguard-rules.pro` — create an empty file (a comment line is fine).

Icons: in Android Studio use *File → New → Image Asset* to generate `ic_launcher` and `ic_launcher_round`.

Wrapper: in the project folder run `gradle wrapper`, or let Android Studio generate it on first sync.

Root `build.gradle.kts` should look like:

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}
```

Test: Gradle sync succeeds, `./gradlew assembleDebug` finishes.

C-02 — Missing import in `Theme.kt` (P1)

Problem: `Color.White` is used but `Color` isn't imported, so compilation fails.

Fix: add at the top of `Theme.kt`:

```
import androidx.compose.ui.graphics.Color
```

Test: project compiles.

C-03 — Invalid time crashes the app (P1)

Problem: The due time is free text. `calculateTriggerTime` uses `SimpleDateFormat.parse`, which throws `ParseException` for input like `abc`. Only `SecurityException` is caught, so the app crashes when you tap Create.

Fix A (safe parsing) in `AlarmScheduler.kt` — replace `calculateTriggerTime` with:

```
import java.time.LocalDate
import java.time.LocalDateTime
import java.time.LocalTime
import java.time.ZoneId
import java.time.format.DateTimeParseException

private fun dueMillis(task: TaskEntity): Long? = try {
    LocalDateTime.of(LocalDate.parse(task.dueDate), LocalTime.parse(task.dueTime))
        .atZone(ZoneId.systemDefault())
        .toInstant()
        .toEpochMilli()
} catch (e: DateTimeParseException) {
    null
}
```

(`java.time` works because `minSdk = 26`.)

Fix B (validate in the dialog) in `QuickAddDialog`:

```
val timeRegex = Regex("^([01]\\d|2[0-3]):[0-5]\\d$")
val timeValid = timeRegex.matches(dueTime)
// on the time field:
OutlinedTextField(
    value = dueTime,
    onValueChange = { dueTime = it },
    label = { Text("Due Time (HH:mm)") },
    singleLine = true,
    isError = !timeValid,
    supportingText = { if (!timeValid) Text("Use 24-hour format, e.g. 17:30") }
)
// confirm button:
onClick = { if (title.isNotBlank() && timeValid) onConfirm(title.trim(), priority, dueTime) }
```

Test: enter `abc`, `25:99`, `5:00` → Create is blocked or shows an error, and the app never crashes. `17:30` works.

C-04 — Reminder silently skipped (P2)

Problem: If the reminder time (due time minus 15 min) is already past, the alarm is skipped with no message. The default due time is a fixed 17:00, so a task added at 16:50 or later gets no reminder.

Fix: replace `scheduleTaskReminder` (full `AlarmScheduler` is under C-05):

```
fun scheduleTaskReminder(task: TaskEntity) {
    if (task.isCompleted) return
    val now = System.currentTimeMillis()
    val due = dueMillis(task) ?: return
    if (due <= now) return                       // already overdue: nothing to remind
    var trigger = due - task.reminderMinutesBefore * 60_000L
    if (trigger <= now) trigger = now + 3_000L   // due soon: remind right away
    scheduleAt(task.id, task.title, task.priority, task.dueTime, trigger)
}
```

In the dialog, default the time to the next hour and reject past times:

```
var dueTime by remember {
    mutableStateOf(java.time.LocalTime.now().plusHours(1).withMinute(0).toString().take(5))
}
// extra validation (today only, until a date picker exists):
val notPast = timeValid && java.time.LocalTime.parse(dueTime).isAfter(java.time.LocalTime.now())
```

Show "That time has already passed today" when `!notPast`. The real fix is a date picker (C-16).

Test: add a task due in 10 min → notification appears within seconds. Add a past time → blocked with a message.

C-05 — Snooze can crash and loses data (P2)

Problem: `ACTION_TASK_SNOOZE` calls `setExactAndAllowWhileIdle` directly. On Android 12+ without exact-alarm permission this throws `SecurityException` inside a receiver. The rescheduled intent also drops priority and due time, so the notification after a snooze shows wrong text.

Fix: move all scheduling into `AlarmScheduler` and reuse it.

Full `AlarmScheduler.kt`:

```
package com.chronos.tasktracker.scheduler

import android.app.AlarmManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build
import com.chronos.tasktracker.data.model.TaskEntity
import com.chronos.tasktracker.receiver.AlarmReceiver
import java.time.LocalDate
import java.time.LocalDateTime
import java.time.LocalTime
import java.time.ZoneId
import java.time.format.DateTimeParseException

class AlarmScheduler(private val context: Context) {
```

```
    private val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager

    fun scheduleTaskReminder(task: TaskEntity) {
        if (task.isCompleted) return
        val now = System.currentTimeMillis()
        val due = dueMillis(task) ?: return
        if (due <= now) return
        var trigger = due - task.reminderMinutesBefore * 60_000L
        if (trigger <= now) trigger = now + 3_000L
        scheduleAt(task.id, task.title, task.priority, task.dueTime, trigger)
    }

    /** Used by task reminders AND by snooze. */
    fun scheduleAt(
        taskId: String, title: String, priority: String,
        dueTime: String, triggerAtMillis: Long
    ) {
        val intent = Intent(context, AlarmReceiver::class.java).apply {
            action = AlarmReceiver.ACTION_TASK_REMINDER
            putExtra(AlarmReceiver.EXTRA_TASK_ID, taskId)
            putExtra(AlarmReceiver.EXTRA_TASK_TITLE, title)
            putExtra(AlarmReceiver.EXTRA_TASK_PRIORITY, priority)
            putExtra(AlarmReceiver.EXTRA_TASK_DUE_TIME, dueTime)
        }
        val pendingIntent = PendingIntent.getBroadcast(
            context, taskId.hashCode(), intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        val canExact = Build.VERSION.SDK_INT < Build.VERSION_CODES.S ||
            alarmManager.canScheduleExactAlarms()
        try {
            if (canExact) {
                alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, triggerAtMillis,
pendingIntent)
            } else {
                alarmManager.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, triggerAtMillis,
pendingIntent)
            }
        } catch (e: SecurityException) {
            alarmManager.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, triggerAtMillis,
pendingIntent)
        }
    }

    fun cancelTaskReminder(taskId: String) {
        val intent = Intent(context, AlarmReceiver::class.java).apply {
            action = AlarmReceiver.ACTION_TASK_REMINDER
        }
        val pendingIntent = PendingIntent.getBroadcast(
            context, taskId.hashCode(), intent,
            PendingIntent.FLAG_NO_CREATE or PendingIntent.FLAG_IMMUTABLE
        )
        if (pendingIntent != null) {
            alarmManager.cancel(pendingIntent)
            pendingIntent.cancel()
```

```
        }
    }

    private fun dueMillis(task: TaskEntity): Long? = try {
        LocalDateTime.of(LocalDate.parse(task.dueDate), LocalTime.parse(task.dueTime))
            .atZone(ZoneId.systemDefault()).toInstant().toEpochMilli()
    } catch (e: DateTimeParseException) {
        null
    }
}
```

In `AlarmReceiver.showNotification`, add two extras to the snooze intent:

```
putExtra(EXTRA_TASK_PRIORITY, priority)
putExtra(EXTRA_TASK_DUE_TIME, dueTime)
```

Replace the whole `ACTION_TASK_SNOOZE` branch with:

```
ACTION_TASK_SNOOZE -> {
    val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    nm.cancel(taskId.hashCode())
    val mins = intent.getIntExtra(EXTRA_SNOOZE_MINUTES, 10)
    AlarmScheduler(context).scheduleAt(
        taskId,
        intent.getStringExtra(EXTRA_TASK_TITLE) ?: "Snoozed Task",
        intent.getStringExtra(EXTRA_TASK_PRIORITY) ?: "medium",
        intent.getStringExtra(EXTRA_TASK_DUE_TIME) ?: "",
        System.currentTimeMillis() + mins * 60_000L
    )
}
```

Test: on Android 12+ with exact-alarm permission denied, tap Snooze → no crash, notification returns in about 10 min with correct priority and time.

C-06 — Tasks tab toggle ignores alarms and `completedAt` (P2)

Problem: Home toggles cancel/reschedule the alarm; the Tasks tab does not, so completed tasks still ring. `completedAt` is never set from the app.

Fix: one shared handler inside `MainAppScreen` (above `Scaffold`):

```
val toggleTask: (TaskEntity) -> Unit = { task ->
    coroutineScope.launch {
        val nowDone = !task.isCompleted
        val now = System.currentTimeMillis()
        val updated = task.copy(
            isCompleted = nowDone,
            completedAt = if (nowDone) now else null,
            updatedAt = now
        )
```

```
        database.taskDao().updateTask(updated)
        if (nowDone) alarmScheduler.cancelTaskReminder(task.id)
        else alarmScheduler.scheduleTaskReminder(updated)
    }
}
```

Use it in both tabs: `TaskRow(task = task, onToggleComplete = { toggleTask(task) })`.

Test: complete a task in the Tasks tab, wait past its time → no notification. Un-complete a future task → notification returns.

C-07 — Notification "Complete" may not save (P2)

Problem: the receiver launches a coroutine without `goAsync()`, so Android can kill the process before the database write finishes.

Fix: replace the `ACTION_TASK_COMPLETE` branch:

```
ACTION_TASK_COMPLETE -> {
    val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    nm.cancel(taskId.hashCode())
    AlarmScheduler(context).cancelTaskReminder(taskId)   // also clears a pending snooze
    val pending = goAsync()
    CoroutineScope(Dispatchers.IO).launch {
        try {
            AppDatabase.getInstance(context).taskDao().markTaskComplete(taskId)
        } finally {
            pending.finish()
        }
    }
}
```

Test: tap Complete on the notification, reopen the app → task is checked.

C-08 — Priority can't be chosen (P2)

Problem: `priority` state exists but has no control, so every task is "medium".

Fix: add inside the dialog's `Column`:

```
Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
    listOf("high", "medium", "low").forEach { p ->
        FilterChip(
            selected = priority == p,
            onClick = { priority = p },
            label = { Text(p.replaceFirstChar { it.uppercase() }) }
        )
    }
}
```

Test: create one task of each priority; rows show the matching colour and text.

C-09 — Home list and progress card (P3)

Problem: "Active Tasks" lists completed tasks too. With no tasks today the bar shows 100%.

Fix in the Home tab:

```
LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
    items(tasks.filter { !it.isCompleted }, key = { it.id }) { task -> /* TaskRow */ }
}
// progress:
progress = { if (todayTasks.isNotEmpty()) completedCount.toFloat() / todayTasks.size else 0f }
// text:
if (todayTasks.isEmpty()) "No tasks due today" else "$completedCount / ${todayTasks.size} Tasks
Completed"
```

Test: with zero tasks the bar is empty; completing a task removes it from Home but keeps it in Tasks.

C-10 — Task ID collision (P2)

Problem: `"task_${System.currentTimeMillis()}"` can repeat on a fast double tap, and `OnConflictStrategy.REPLACE` would silently overwrite the first task. Create can also fire twice before the dialog closes.

Fix in `onConfirm`:

```
onConfirm = { title, priority, dueTime ->
    showAddDialog = false                       // close first: blocks double taps
    coroutineScope.launch {
        val newTask = TaskEntity(
            id = java.util.UUID.randomUUID().toString(),
            title = title, priority = priority,
            dueDate = todayStr, dueTime = dueTime
        )
        database.taskDao().insertTask(newTask)
        alarmScheduler.scheduleTaskReminder(newTask)
    }
}
```

(`todayStr` can be `java.time.LocalDate.now().toString()`.)

Test: double-tap Create quickly → exactly one task.

C-11 — Destructive migration (P2, fix before release)

Problem: `fallbackToDestructiveMigration()` deletes every user task whenever you change the database version.

Fix: keep it while developing, but before any real release:

```
Room.databaseBuilder(context.applicationContext, AppDatabase::class.java, "chronos_database")
    .addMigrations(/* MIGRATION_1_2, ... */)
    .build()
```

Set `exportSchema = true` and add to `app/build.gradle.kts` inside `ksp`'s config: `arg("room.schemaLocation", "$projectDir/schemas")`.

Test: install an old build, add tasks, install the new build over it → tasks remain.

C-12 — Exact-alarm permission (P2)

Problem: nothing asks the user to allow exact alarms, so on newer Android versions reminders quietly become inexact. `USE_EXACT_ALARM` is meant for alarm-clock/calendar apps and may be rejected at Play Store review.

Fix:

1. In the manifest, remove `USE_EXACT_ALARM` and keep `SCHEDULE_EXACT_ALARM`.
2. In `MainActivity.onCreate`, after the notification permission block:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    val am = getSystemService(android.app.AlarmManager::class.java)
    if (!am.canScheduleExactAlarms()) {
        startActivity(
            Intent(
                android.provider.Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM,
                android.net.Uri.parse("package:$packageName")
            )
        )
    }
}
```

(add `import android.content.Intent`). Better UX later: explain why first, then open settings.

Test: on Android 12+ with permission off, the settings screen opens; after allowing it, reminders fire on time.

C-13 — Alarms lost after update or timezone change (P2)

Problem: alarms can be cleared when the app is updated, and tasks stored as local date/time drift when the timezone or clock changes. `BootReceiver` only handles reboot.

Fix: add to the `BootReceiver` intent-filter in the manifest:

```
<action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
<action android:name="android.intent.action.TIMEZONE_CHANGED" />
<action android:name="android.intent.action.TIME_SET" />
```

and extend the check in `BootReceiver.onReceive`:

```
val actions = setOf(
    Intent.ACTION_BOOT_COMPLETED,
    "android.intent.action.QUICKBOOT_POWERON",
    Intent.ACTION_MY_PACKAGE_REPLACED,
    Intent.ACTION_TIMEZONE_CHANGED,
    Intent.ACTION_TIME_CHANGED
)
if (intent.action in actions) { /* existing reschedule code */ }
```

Test: adb: `adb shell am broadcast -a android.intent.action.TIMEZONE_CHANGED -p com.chronos.tasktracker`, then check pending alarms with `adb shell dumpsys alarm | grep chronos`.

C-14 — Developer badge (P3)

Fix: delete the `Spacer` and the `Surface { Text("Native Compose" ...) }` block from the top bar in `MainActivity.kt`.

C-15 — README overclaims, unused dependency (P3)

Problem: the README claims recurrence, subtasks, notes, and "background sync". The entity has fields for them but no logic or UI exists, and `work-runtime-ktx` is unused.

Fix: either reword the README to list only what works (tasks, local storage, exact reminders, snooze/complete, reboot restore), or build the features. Remove `implementation(libs.androidx.work.runtime.ktx)` until it's needed.

C-16 — No delete, edit, or empty state (P2)

Fix (delete with undo):

```
val snackbarHostState = remember { SnackbarHostState() }
// Scaffold(snackbarHost = { SnackbarHost(snackbarHostState) }, ...)

val deleteTask: (TaskEntity) -> Unit = { task ->
    coroutineScope.launch {
        alarmScheduler.cancelTaskReminder(task.id)
        database.taskDao().deleteById(task.id)
        val result = snackbarHostState.showSnackbar("Task deleted", actionLabel = "Undo")
        if (result == SnackbarResult.ActionPerformed) {
            database.taskDao().insertTask(task)
            alarmScheduler.scheduleTaskReminder(task)
        }
    }
}
```

Add `onDelete: () -> Unit` to `TaskRow` with:

```
IconButton(onClick = onDelete) {
    Icon(Icons.Default.Delete, contentDescription = "Delete task")
}
```

Empty state: when the list is empty, show `Text("No tasks yet. Tap + to add one.", color = TextSecondary)`.
Next: edit dialog (reuse `QuickAddDialog` with initial values) and a Material3 `DatePicker` for the due date.

C-17 — Accessibility (P3)

- The checkbox has no label. Add (with `import androidx.compose.ui.semantics.*`):
  `modifier = Modifier.semantics { contentDescription = "Mark ${task.title} complete" }`
- Raise the 11 sp due/priority line to at least 12 sp.
- Notification uses `android.R.drawable` icons. Replace them with your own monochrome icons.
- `setVibrate(...)` on the notification builder is ignored on Android 8+. The channel controls vibration, so remove it.

Fix order

1. C-01, C-02 (get it to compile)
2. C-03, C-04, C-05, C-06, C-07 (crashes and wrong alarms)
3. C-08, C-10, C-09 (basic correctness)
4. C-12, C-13, C-11 (reliability before release)
5. C-16, C-17, C-14, C-15 (polish and features)

Regression checklist (run after each group)

- [ ] App builds and launches; no crash on first open.
- [ ] Create a task with each priority; it persists after force-close and reopen.
- [ ] Invalid time (`abc`, `25:99`) is rejected without a crash.
- [ ] Task due in 10 min → notification appears; Complete and Snooze both work.
- [ ] Complete a task in each tab → no later notification; un-complete → reminder returns.
- [ ] Reboot the phone → pending reminders still fire.
- [ ] Change timezone → reminders still fire at the right local time.
- [ ] Double-tap Create → one task only.
- [ ] Delete then Undo restores the task and its reminder.

Not verified

None of the above was run on a device. The web app, performance, security of the web build, and competitor comparison are still pending.